Projeto IoT completo com **ESP32 + AWS + MQTT + Node-RED + MySQL + Dashboard Web em tempo real**.

1 - Arquitetura Geral do Projeto

Dispositivo

- ESP32
    - Lê sensores
    - Publica dados via MQTT
    - Assina tópico MQTT para receber comandos

Servidor (AWS EC2 Ubuntu)

- Apache (Usado como Proxy)
- Mosquitto (Broker MQTT)
- Node-RED
- MySQL
- Dashboard Web (Node-RED Dashboard)

1.1 - ETAPA 1 – Infraestrutura AWS (Base)

- Criar instância Ubuntu EC2
- Associar IP elástico
- Instalar Apache
- Security Group liberar portas:

  22 (SSH)
  80 (HTTP)
  1883 (MQTT)
  1880 (Node-RED) (Será bloqueada no final)
  3306 (MySQL – opcional externo)

1.2 - ETAPA 2 – Instalar Broker MQTT (Mosquitto)

Instalar:

- mosquitto
- mosquitto-clients

Configurar:

- Habilitar autenticação
- Criar usuário/senha
- Liberar listener externo

- Configurar como serviço
- (Recomendado depois: habilitar TLS)

Testar:

- Publicar e assinar tópico via terminal (usado terminal e MQTTBox)

1.3 - ETAPA 3 – Instalar MySQL

Instalar:

- mysql-server
- Configurar como serviço

Criar:

- Banco: iot_db
- Tabela: sensor_data
- Usuário específico para Node-RED

```
+-------------+-------------+------+-----+-------------------+-------------------+
| Field       | Type        | Null | Key | Default           | Extra             |
+-------------+-------------+------+-----+-------------------+-------------------+
| id          | int         | NO   | PRI | NULL              | auto_increment    |
| device_id   | varchar(50) | YES  |     | NULL              |                   |
| temperatura | float       | YES  |     | NULL              |                   |
| umidade     | float       | YES  |     | NULL              |                   |
| status      | varchar(20) | YES  |     | NULL              |                   |
| timestamp   | datetime    | YES  |     | CURRENT_TIMESTAMP |                   |
+-------------+-------------+------+-----+-------------------+-------------------+
```

1.4 - ETAPA 4 – Instalar Node-RED

Instalar:

- Node.js
- Node-RED

Configurar:

- Rodar como serviço (systemd)
- Instalar nodes adicionais:
    - node-red-dashboard 2.0
    - node-red-node-mysql

1.5 - ETAPA 5 – Criar Fluxo no Node-RED

Parte A – Receber dados MQTT

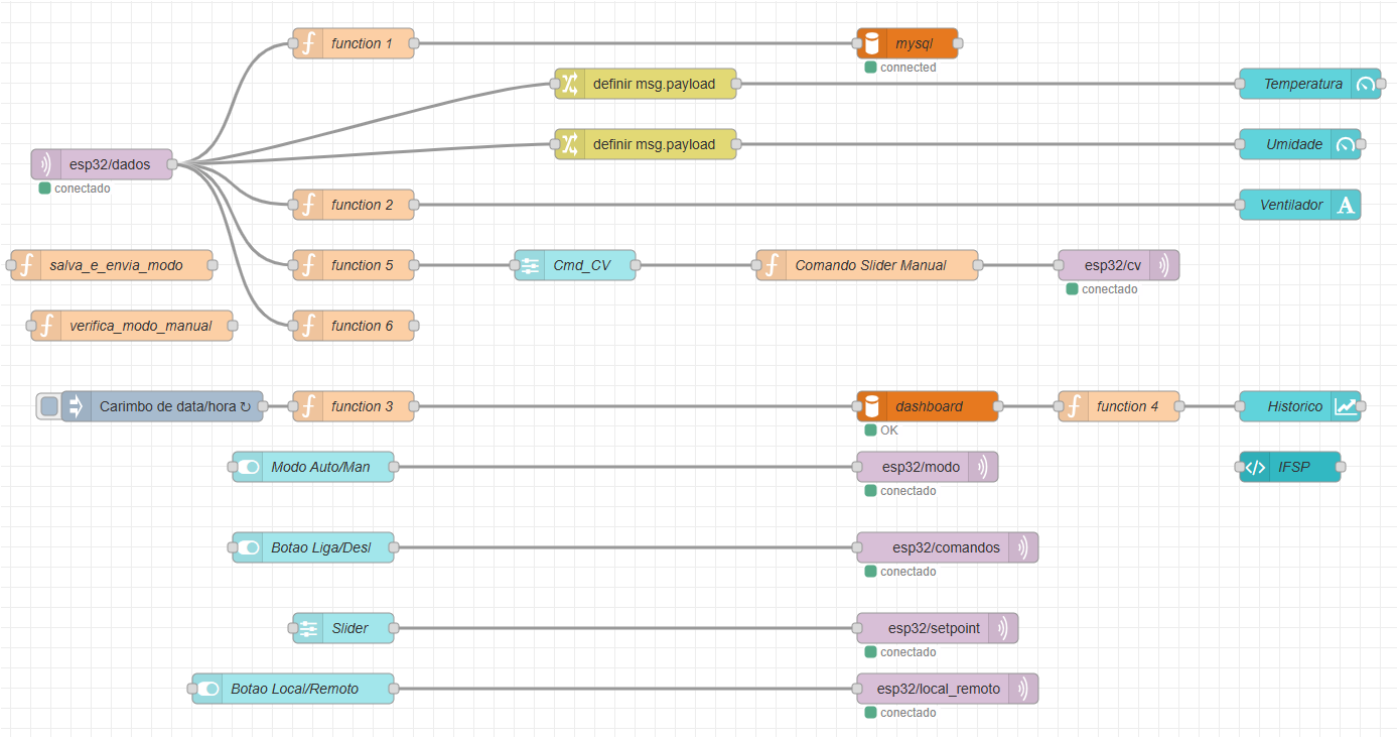
- Node MQTT In
- Converter JSON
- Inserir no MySQL

Parte B – Dashboard em tempo real

- Gauge (temperatura)
- Chart (histórico)
- Status indicador
- Botão ON/OFF

Parte C – Enviar comando ao ESP32

- Botão no dashboard
- MQTT Out
- Publicar tópico:



≡ HOME

INSTITUTO FEDERAL
São Paulo

Curso de Pós-graduação em IoT

Plataformas de Prototipação para Internet das Coisas

Trabalho: Servidor em Nuvem, Apache, Node-RED, IoT

Professor: Marcos Aparecido Chaves Ferreira

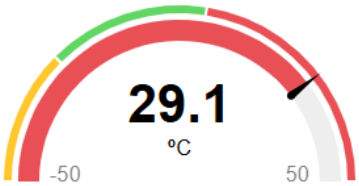
Aluno: Moisés

Utilize o menu lateral para acessar o painel SCADA e os gráficos de histórico do sistema.

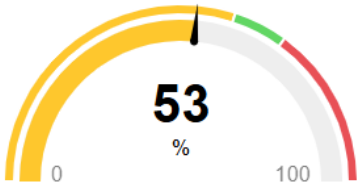
≡ PAINEL-IOT

CAMARA-FRIA

Temperatura:



Umidade:



Ventilador: **LIGADO**

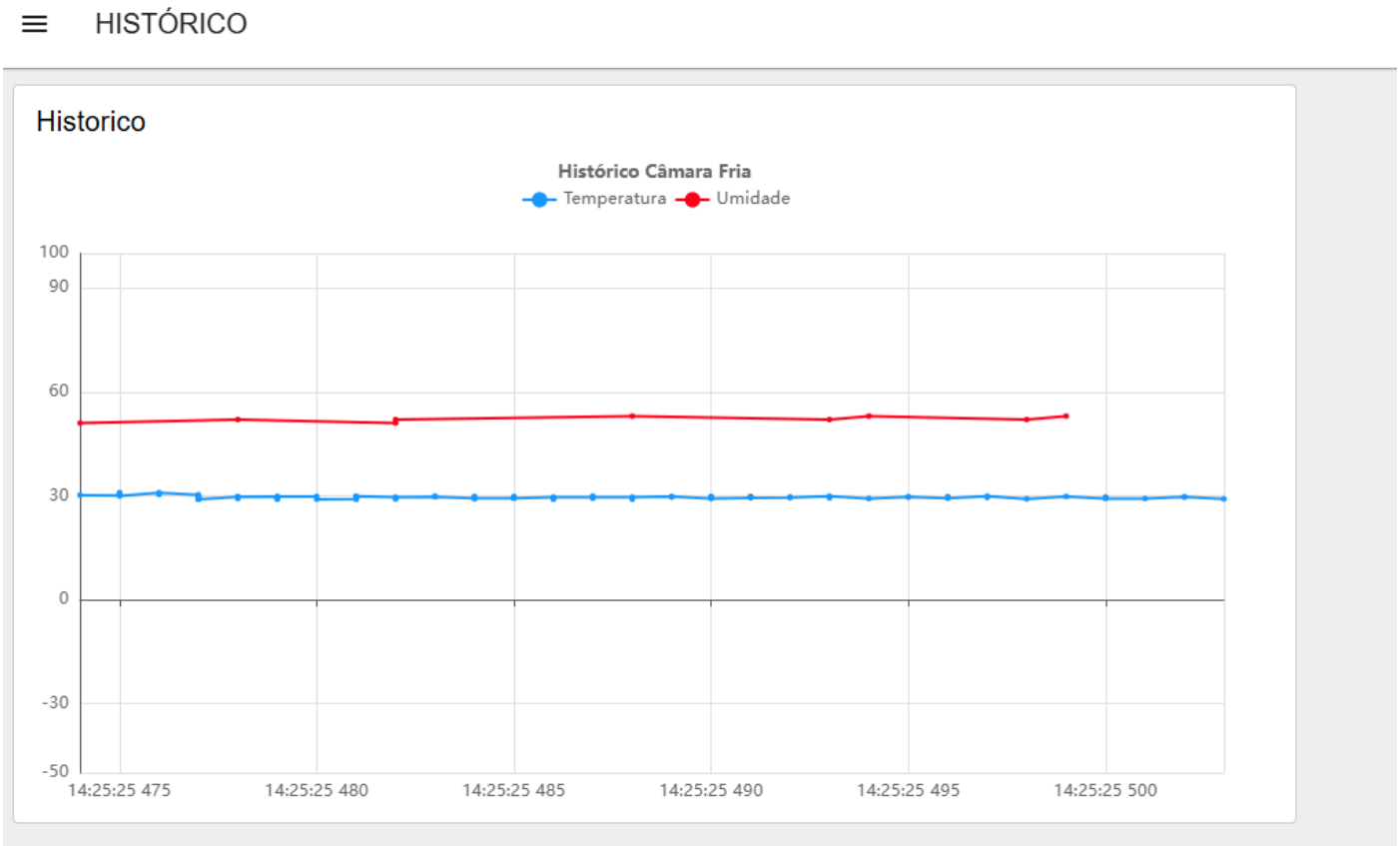
SP Temperatura: 28

Variavel de Controle 100

Ventilador: Desliga / Liga

Modo: Local / Remoto

Malha: Manual / Auto



1.6 - ETAPA 6 – Programação do ESP32

No ESP32:

Bibliotecas:

- WiFi.h
- PubSubClient.h
- DH.h
- Wire.h
- Adafruit_GFX.h
- Adafruit_SSD1306.h
- ArduinoJson.h

Funções:

- Conectar WiFi
- Conectar MQTT
- Publicar:

2 – Desenvolvimento lógica e hardware.

Desenvolvimento de uma automação baseado no microcontrolador ESP32-S3, integrado a um painel de supervisão em nuvem via Node-RED e MQTT (AWS). O sistema lê temperatura e umidade, processa uma malha de controle PID e aciona um inversor simulado por um Micro Servo SG90, respeitando lógicas de segurança industriais.

2.1 - Desafios de Hardware e Soluções

Durante a integração mecânica, o servo motor apresentou anomalias (vibrações e saltos bruscos). O diagnóstico revelou problemas elétricos, não de software.

- Problema de Corrente (Amperagem): O ESP32 não consegue fornecer a corrente de pico necessária para tirar o motor da inércia, causando quedas de tensão no barramento de 3.3V.
    - Solução: Implementação de uma fonte de alimentação externa de 5V dedicada exclusivamente ao atuador.
- Problemas de Incompatibilidade: O microcontrolador entrava modo fault quando usava display OLED e biblioteca Esp32Servo.h .
    - Solução: Criar lógica de controle sem uso da biblioteca.

2.2 - Desafios de Software e Arquitetura

A transição de um simples acionamento de código para uma arquitetura de nível industrial exigiu refinamentos pesados no firmware.

2.3 - Rampa de Suavização (Filtro Passa-Baixa Matemático)

Para evitar que comandos instantâneos da rede causassem trancos mecânicos e picos de corrente no inversor, o controle de posição foi desvinculado do comando imediato.

- Ação: Criação de uma rampa linear flutuante. O motor recebe atualizações a cada 15 milissegundos, limitando seu incremento a 0.5%. O atuador leva exatamente 3 segundos para ir de 0 a 100%, garantindo o efeito de "Transferência Suave" característico de inversores industriais.

2.4 - Intertravamento de Segurança

Depender da nuvem para bloqueios críticos provou-se ineficaz devido a gargalos e dessincronização de pacotes.

- Ação: A única fonte da verdade passou a ser o ESP32.
    - Regra 1 (Local/Remoto): Comandos operacionais vindos do MQTT (Node-RED) são sumariamente descartados no callback se a chave física do painel estiver em modo "Local".
    - Regra 2 (Ventilador): Se a variável status (motor) for falsa, a variável de controle físico da válvula é forçada a 0%, independentemente do valor do CV da malha.

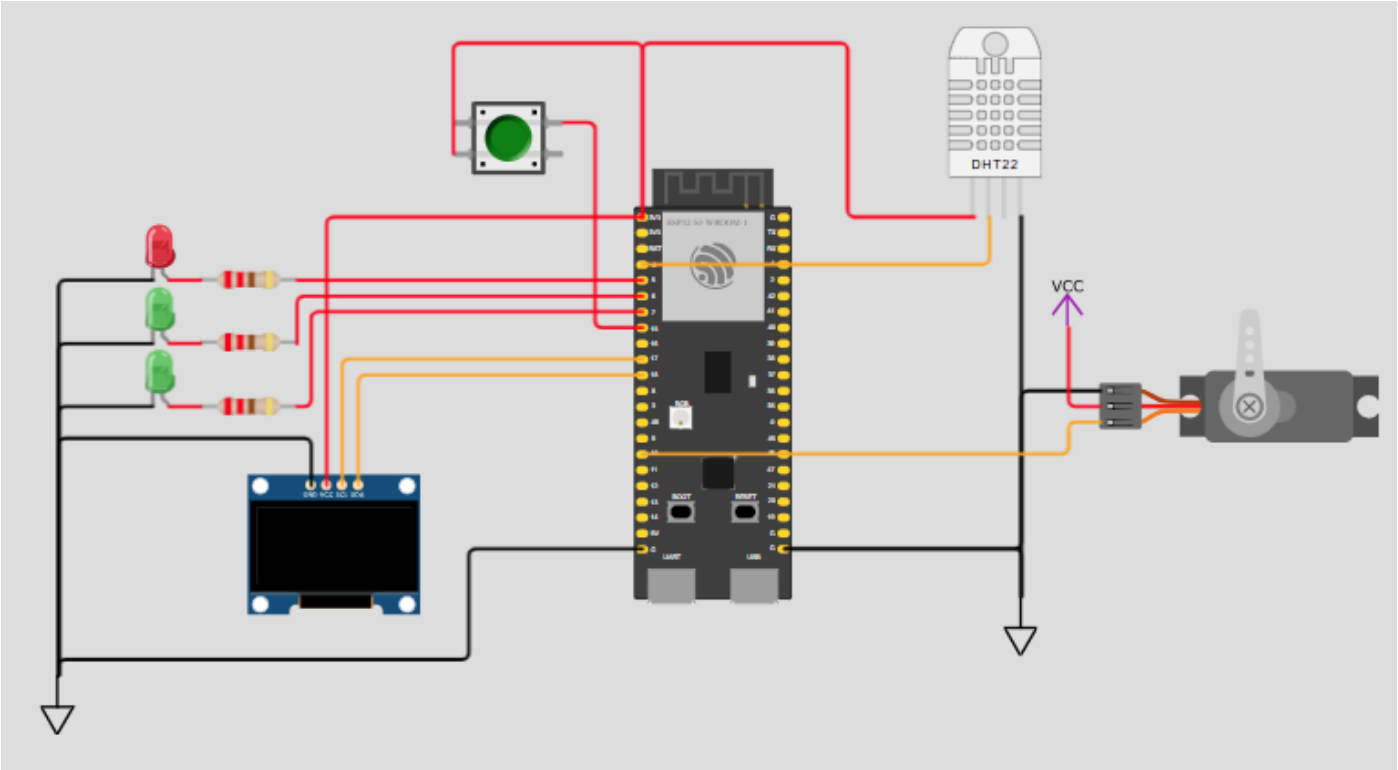
2.5 - Controle PID Profissional

O projeto não utiliza apenas as fórmulas matemáticas puras, mas implementou defesas industriais para a estabilidade da planta térmica:

- Fórmulas Utilizadas (Resfriamento): Erro = Temperatura - Setpoint Proporcional (P) = Kp * Erro Integral (I) = I + (Ki * Erro * Tempo) Derivativo (D) = Kd * (Erro - ErroAnterior) / Tempo Controle (CV) = P + Integral + D
- Proteção Anti-Windup: Travamento do limite da memória Integral entre 0 e 100 para impedir o acúmulo infinito do erro quando o inversor já saturou fisicamente.

- Bumpless Transfer (Transferência Suave): Quando a malha está em "Manual", a variável Integral é constantemente igualada ao CV digitado pelo operador. Ao passar para "Automático", o PID assume o processo sem solavancos.

2.6 – Diagrama elétrico



2.7 - Código Fonte Definitivo (C++)

Em anexo.